CodeQuotient

Department of Computer Science & Engineering

LAB MANUAL

22CS025 - Algorithm Design & Implementation-5Sem-2022

Student Name	**JOEL MATTHEW**
Email	**joel465.be22@chitkara.edu.in**
Course Name	**22CS025 - Algorithm Design & Implementation-5Sem-2022**

Faculty Incharge

_____

Aim: Longest Common Subsequence (LCS)

Overlapping Subproblems and Optimal Substructure properties are the prerequisites for solving a problem with dynamic programming. In this problem you need to figure them out. LCS is a classic computer science problem, the basis of diff (a file comparison program that outputs the differences between two files), and has applications in bioinformatics.
A subsequence is defined as an ordered subset of the array's elements having the same sequential ordering as the original array.
Given two sequences, find the length of longest subsequence present in both of them. For example,
For sequences "**ABAZDC**" and "**BACBAD**" the LCS is "**ABAD**" of length 4.
For sequences "**AGGTAB**" and "**GXTXAYB**" the LCS is "**GTAB**" of length 4.
Note: A string of length n has 2^n-1 different possible subsequences, which implies that the time complexity of the brute force approach will be O(n * 2^n). It takes O(n) time to check if a subsequence is common to both the strings. This time complexity can be improved using dynamic programming.

Input Format

The first line of input contains an integer T denoting the no of test cases. Then T test cases follow.
Each test case contains two strings in two lines.

Output Format

For each test case, print the length of the Longest Common Subsequence in new lines.

Sample Input

2
ABAZDC
BACBAD
AGGTAB
GXTXAYB

Sample Output

4
4

Solution:

```
class Result
{
  static int longestCommonSubsequence(String str1, String str2){
    int m = str1.length() ;
    int n = str2.length() ;
    int[][] dp = new int[m+1][n+1] ;
    for(int i = 1 ; i <= m ;i++){
        for(int j = 1 ; j <= n ; j++) {
            if(str1.charAt(i-1) == str2.charAt(j-1)) {
                dp[i][j] = dp[i-1][j-1] + 1;
            }
            else{
                dp[i][j] = Math.max(dp[i-1][j],dp[i][j-1]) ;
            }
        }
    }
      return dp[m][n] ;
```


JOEL MATTHEW
joel465.be22@chitkara.edu.in

```
    }
}
```


JOEL MATTHEW
joel465.be22@chitkara.edu.in